MCI lab7 Report

Ali Hussain Zaidi (sz10384)

Haider Zaidi (mz10270)

Task#1

Code Snippets:

```
static void MX_ADC1_Init(void)
{
    ADC_MultiModeTypeDef multimode = {0};
    ADC_ChannelConfTypeDef sConfig = {0};

    /* USER CODE BEGIN ADC1_Init 1 */

    /* USER CODE END ADC1_Init 1 */

    /** Common config
    */
    hadc1.Instance = ADC1;
    hadc1.Init.ClockPrescaler = ADC_CLOCK_ASYNC_DIV1;
    hadc1.Init.Resolution = ADC_RESOLUTION_12B;
    hadc1.Init.ScanConvMode = ADC_SCAN_DISABLE;
    hadc1.Init.ContinuousConvMode = DISABLE;
    hadc1.Init.DiscontinuousConvMode = DISABLE;
    hadc1.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
    hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;
    hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
    hadc1.Init.NbrOfConversion = 1;
    hadc1.Init.DMAContinuousRequests = DISABLE;
    hadc1.Init.EOCSelection = ADC_EOC_SINGLE_CONV;
    hadc1.Init.LowPowerAutoWait = DISABLE;
    hadc1.Init.Overrun = ADC_OVR_DATA_OVERWRITTEN;
    if (HAL_ADC_Init(&hadc1) != HAL_OK)
    {
        Error_Handler();
    }

    /** Configure the ADC multi-mode
    */
    multimode.Mode = ADC_MODE_INDEPENDENT;
    if (HAL_ADCEx_MultiModeConfigChannel(&hadc1, &multimode) != HAL_OK)
    {
        Error_Handler();
    }
```

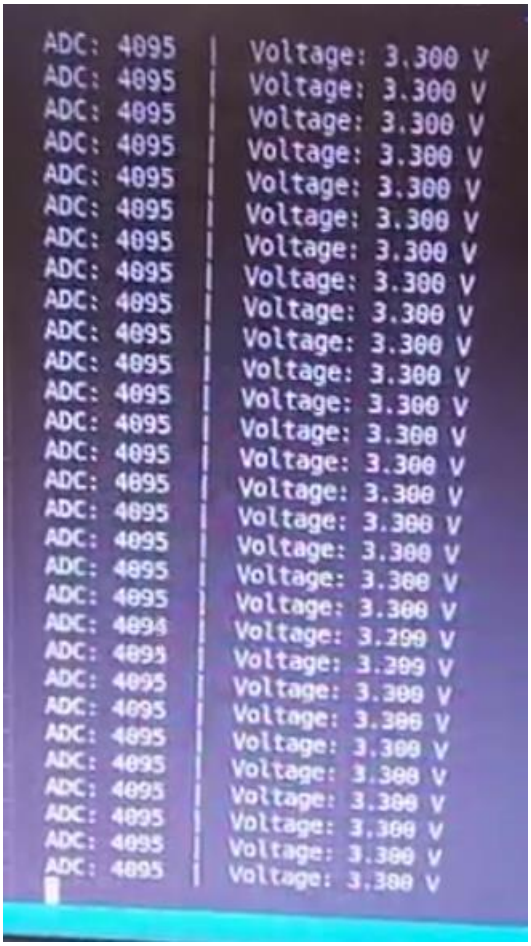
```c
    if (HAL_ADC_Init(&hadc1) != HAL_OK)
    {
        Error_Handler();
    }

    /** Configure the ADC multi-mode
    */
    multimode.Mode = ADC_MODE_INDEPENDENT;
    if (HAL_ADCEx_MultiModeConfigChannel(&hadc1, &multimode) != HA
    {
        Error_Handler();
    }

    /** Configure Regular Channel - ADC1_IN1 on PA1
    */
    sConfig.Channel = ADC_CHANNEL_1;  // PA1 is ADC1_IN1
    sConfig.Rank = ADC_REGULAR_RANK_1;
    sConfig.SamplingTime = ADC_SAMPLETIME_601CYCLES_5;  // Longer
    sConfig.SingleDiff = ADC_SINGLE_ENDED;
    if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
    {
        Error_Handler();
    }
```

```c
uint32_t Read_ADC_Averaged(void)
{
    uint32_t sum = 0;

    // Take multiple samples
    for (int i = 0; i < ADC_BUFFER_SIZE; i++)
    {
        sum += Read_ADC();
        HAL_Delay(1);  // Small delay between samples
    }

    return sum / ADC_BUFFER_SIZE;
}

/**
  * @brief Print ADC value with voltage (using integer math to avoid float issues)
  */
void Print_ADC_Value(uint32_t adc_value)
{
    // Calculate voltage in millivolts: (adc_value * 3300) / 4095
    uint32_t voltage_mv = (adc_value * ADC_REF_VOLTAGE) / ADC_MAX_VALUE;

    // Split into volts and millivolts for display
    uint32_t volts = voltage_mv / 1000;
    uint32_t millivolts = voltage_mv % 1000;

    UART_Printf("ADC: %4lu  |  Voltage: %lu.%03lu V\r\n",
                adc_value, volts, millivolts);
}

/* USER CODE END 0 */
```

```c
int main(void)
{
    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration--------------------------------------------------------*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_ADC1_Init();
    MX_I2C1_Init();
    MX_SPI1_Init();
    MX_USART2_UART_Init();
    MX_USB_PCD_Init();

    /* USER CODE BEGIN 2 */

    // Calibrate ADC (optional but recommended)
    HAL_ADCEx_Calibration_Start(&hadc1, ADC_SINGLE_ENDED);

    // Print header
    UART_Printf("\033[2J\033[H"); // Clear screen
    UART_Printf("========================================\r\n");
    UART_Printf("   LAB 7 TASK 1: ADC SENSOR READING\r\n");
    UART_Printf("========================================\r\n\n");
    UART_Printf("Potentiometer on PA1 (ADC1_IN1)\r\n");
    UART_Printf("ADC: 12-bit (0-4095) | Vref: 3.3V\r\n\n");
    UART_Printf("Reading every 500ms...\r\n\n");
```

```c
    while (1)
    {
        /* USER CODE END WHILE */

        /* USER CODE BEGIN 3 */

        // Read ADC value (averaged for stability)
        uint32_t adc_value = Read_ADC_Averaged();

        // Print the value with voltage
        Print_ADC_Value(adc_value);

        // Wait before next reading
        HAL_Delay(50);
    }
    /* USER CODE END 3 */
}
```

Output



```
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4094 | Voltage: 3.299 V
ADC: 4095 | Voltage: 3.299 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
ADC: 4095 | Voltage: 3.300 V
```

Task#2:

Code Snippets:

```c
#include "main.h"
#include <stdio.h>
#include <string.h>
#include <stdarg.h>
#include <stdbool.h>

/* CMSIS DSP includes */
#include "arm_math.h"
```

```c
// ADC sample buffers
static volatile uint16_t adc_raw_buffer[SAMPLE_COUNT];
static volatile float32_t adc_float_buffer[SAMPLE_COUNT];
static volatile float32_t filtered_buffer[SAMPLE_COUNT];

// Moving average filter buffer
static float32_t filter_buffer[FILTER_LEN];
static uint8_t filter_index = 0;

// Sampling control
static volatile uint16_t sample_index = 0;
static volatile uint8_t sampling_complete = 0;
static volatile uint32_t sampling_start_time = 0;

/* USER CODE END PV */

/* Private function prototypes -----------------------------------------
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_ADC1_Init(void);
static void MX_I2C1_Init(void);
static void MX_SPI1_Init(void);
static void MX_USB_PCD_Init(void);
static void MX_USART2_UART_Init(void);
```

```c
float32_t Moving_Average_Filter(float32_t new_sample)
{
    float32_t sum = 0.0f;

    // Add new sample to buffer
    filter_buffer[filter_index] = new_sample;
    filter_index = (filter_index + 1) % FILTER_LEN;

    // Calculate average of all samples in buffer
    for (int i = 0; i < FILTER_LEN; i++)
    {
        sum += filter_buffer[i];
    }

    return sum / FILTER_LEN;
}
```

```c
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    if (hadc->Instance == ADC1)
    {
        // Store raw ADC value
        adc_raw_buffer[sample_index] = HAL_ADC_GetValue(hadc);

        // Convert to float for filtering (0.0 - 3.3V)
        adc_float_buffer[sample_index] = (adc_raw_buffer[sample_index] * 3.3f) / 4095.0f;

        sample_index++;

        // Check if we've collected enough samples
        if (sample_index >= SAMPLE_COUNT)
        {
            sampling_complete = 1;
            HAL_ADC_Stop_IT(hadc);
        }
        else
        {
            // Start next conversion
            HAL_ADC_Start_IT(hadc);
        }
    }
}
```

```c
void Apply_Filter_to_Buffer(void)
{
    // Reset filter
    Moving_Average_Init();

    // Apply filter to each sample
    for (int i = 0; i < SAMPLE_COUNT; i++)
    {
        filtered_buffer[i] = Moving_Average_Filter(adc_float_buffer[i]);
    }
}
```

```c
void Send_Data_to_Python(void)
{
    for (int i = 0; i < SAMPLE_COUNT; i++)
    {
        // Convert ADC raw value to voltage (mV) for better plotting
        uint32_t raw_mv = (adc_raw_buffer[i] * ADC_REF_VOLTAGE) / ADC_MAX_VALUE;
        uint32_t filtered_mv = (uint32_t)(filtered_buffer[i] * 1000);  // Convert V to mV

        UART_Printf("%lu,%lu\r\n", raw_mv, filtered_mv);
    }
}
```

```c
    while (1)
    {
        /* USER CODE END WHILE */

        /* USER CODE BEGIN 3 */

        // Start ADC sampling in interrupt mode
        ADC_Start_Continuous();

        // Wait for sampling to complete
        while (!sampling_complete)
        {
            HAL_Delay(1);
        }

        // Calculate actual sampling rate
        uint32_t sampling_time = HAL_GetTick() - sampling_start_time;
        float sampling_rate = (SAMPLE_COUNT * 1000.0f) / sampling_time;

        UART_Printf("\r\nCollected %d samples in %lu ms\r\n",
                    SAMPLE_COUNT, sampling_time);
        UART_Printf("Sampling rate: %.2f Hz\r\n\n", sampling_rate);

        // Apply filter to the buffer
        Apply_Filter_to_Buffer();

        // Send data to Python for plotting
        UART_Printf("=== DATA START ===\r\n");
        Send_Data_to_Python();
        UART_Printf("=== DATA END ===\r\n\n");

        // Wait before next acquisition
        UART_Printf("Waiting 2 seconds before next acquisition...\r\n\n");
        HAL_Delay(2000);
    }
    /* USER CODE END 3 */
```

```c
void Moving_Average_Init(void)
{
    filter_index = 0;

    for (int i = 0; i < FILTER_LEN; i++)
    {
        filter_buffer[i] = 0.0f;
    }
}
```

Outputs:

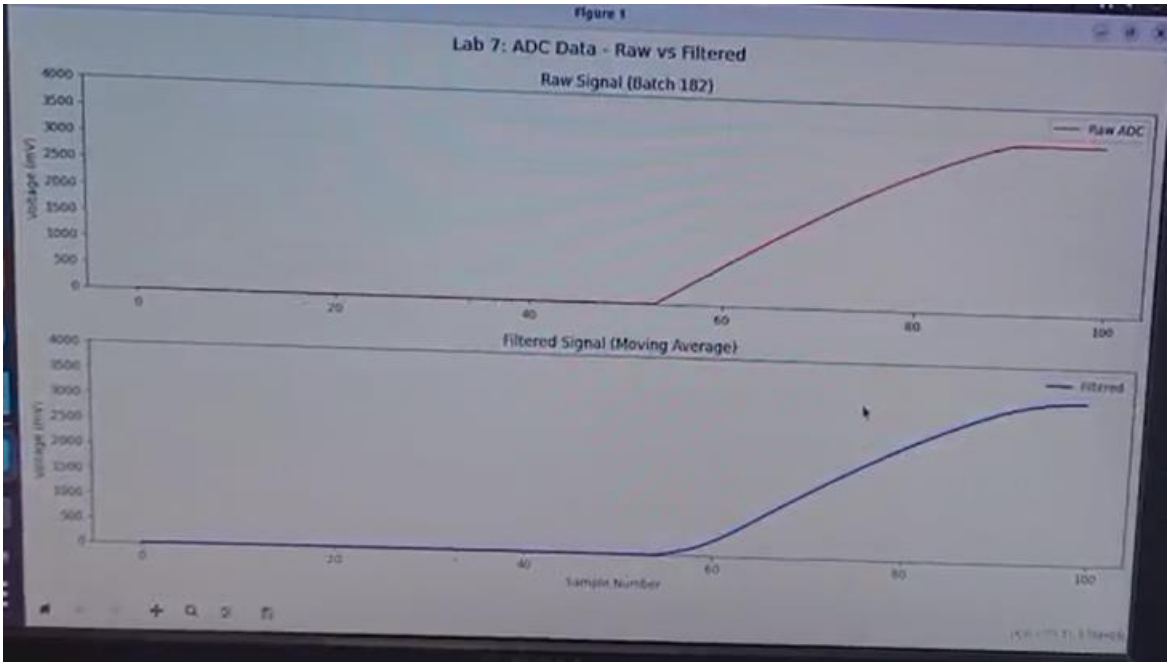
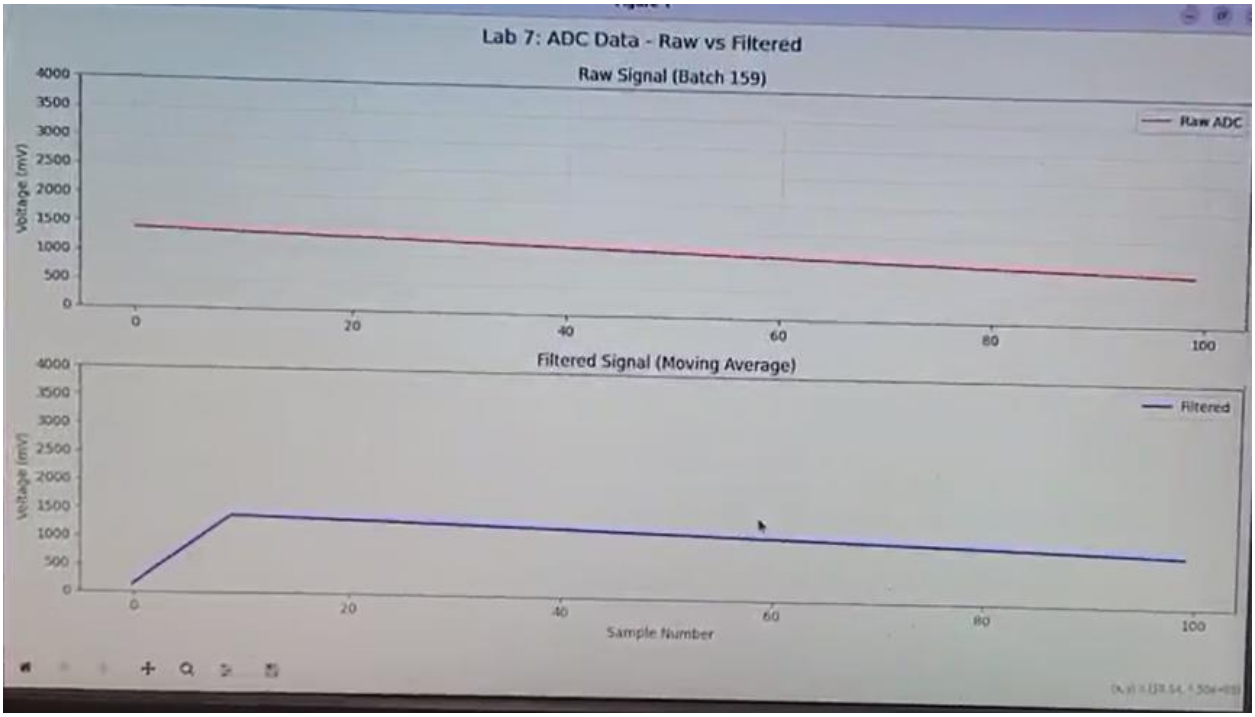


Figure 1
Lab 7: ADC Data - Raw vs Filtered
Raw Signal (Batch 182)
Raw ADC
Voltage (mV)
Filtered Signal (Moving Average)
Filtered
Sample Number


Lab 7: ADC Data - Raw vs Filtered
Raw Signal (Batch 159)
Raw ADC
Voltage (mV)
Filtered Signal (Moving Average)
Filtered
Sample Number

Helper file used for plotting:

```python
# Check for data markers
if line == "=== DATA START ===":
    print(f"\n▶ New data batch started at {time.strftime('%H:%M:%S')}")
    raw_data = []
    filtered_data = []
    sample_numbers = []
    cnt = 0
    in_data_block = True
    data_count += 1
    continue
```

```python
# Parse data if we're in a block
if in_data_block:
    try:
        # Split the line
        values = line.split(',')
        if len(values) == 2:
            raw = int(values[0])
            filtered = int(values[1])

            raw_data.append(raw)
            filtered_data.append(filtered)
            sample_numbers.append(cnt)
            cnt += 1

            # Print progress every 10 samples
            if cnt % 10 == 0:
                print(f"  Received {cnt} samples...", end='\r')

    except ValueError as e:
        print(f"  Invalid data: {line}")
    except Exception as e:
        print(f"  Parse error: {e}")
```

```python
finally:
    # Clean up
    if 'ser' in locals():
        ser.close()
        print("✓ Serial port closed")

    # Keep plot window open
    plt.ioff()
    plt.show()

print("\nDone!")
```

```python
if cnt > 0:
    # Clear axes
    ax1.clear()
    ax2.clear()

    # Plot raw data
    ax1.plot(sample_numbers, raw_data, 'r-', linewidth=1.5, label='Raw ADC')
    ax1.set_ylabel('Voltage (mV)')
    ax1.set_title(f'Raw Signal (Batch {data_count})')
    ax1.grid(True, alpha=0.3)
    ax1.legend(loc='upper right')
    ax1.set_ylim(0, 4000)

    # Plot filtered data
    ax2.plot(sample_numbers, filtered_data, 'b-', linewidth=2, label='Filtered')
    ax2.set_xlabel('Sample Number')
    ax2.set_ylabel('Voltage (mV)')
    ax2.set_title('Filtered Signal (Moving Average)')
    ax2.grid(True, alpha=0.3)
    ax2.legend(loc='upper right')
    ax2.set_ylim(0, 4000)

    # Update plot
    plt.tight_layout()
    plt.draw()
    plt.pause(0.1)

    # Print statistics
    if len(raw_data) > 0:
        print(f"  Raw range: {min(raw_data)}-{max(raw_data)} mV")
        print(f"  Filtered range: {min(filtered_data)}-{max(filtered_data)} mV")
continue
```

```python
# USING THE CORRECT PORT!
PORT = '/dev/ttyUSB0'  # Changed from ttyACM0 to ttyUSB0
BAUD = 115200
TIMEOUT = 2
```

Question : (a) Explain the relationship between ADC resolution and quantization error. Why does increasing resolution reduce error?

Answer:

As ADC resolution increases, quantization error decreases. Quantization error is the small difference between the actual analog input and its digital value because the signal is rounded to the nearest level. The maximum error is ±½ LSB, where LSB = Vref / 2^n. When the resolution increases, the number of levels increases and the step size becomes smaller, giving a more accurate representation of the signal. For example, with a 3.3V reference, moving from 8-bit to 12-bit reduces the step size from about 12.9 mV to 0.8 mV, which also reduces the maximum error significantly.